Milestone 4

Model Training

Contents:

- Overview / Objective
- Architecture
- Chat Flow (flow chart)
- Hyperparameter tuning for policy RAG
    - Scoring
    - Observation
- Hyperparameter tuning for product RAG
    - Scoring
    - Observation

Overview / Objective

- This fourth milestone focuses primarily on the policy and product RAGs, their hyperparameter tuning, observations and scoring their outputs, as outlined in the previously demonstrated flowchart. It builds upon the work from earlier milestones to integrate all components — from data preprocessing to query handling and product recommendation — into a functional end-to-end system for the RAG part, with a basic frontend UI for the ChatBot..
- This document provides:
    - Detailed information on the **dataset** used for catalog, customer, and transaction data.
    - A description of the **system architecture** and its workflow.
    - The **initial results** obtained after implementing the architecture.
    - Hyperparameter tuning for policy and product RAGs.

    - **Observations** and notes for further improvement in the next milestone.
    - An **update on changes** made since the previous milestone.
    - The **division of work** among team members for this stage.
    - And finally, a **summary of the previous milestones**, including data preprocessing and architecture design progress.
- **Link to Previous Milestones** [Link]

Architecture

Login and Log out:

- To login, the user must give a prompt in the ChatBot in this format "cust_id:{customer_id}". e.g. – "cust_id:1", this will lead to logging in as customer 1.
- To log out, the user needs to type "logout" in the ChatBot.

Context:

Must be built before answering any user query. If context is missing, the system MUST ask for it — it cannot proceed.

The context contains all the user details which are available in our database. This is maintained to get all the details about the customer in 1 place, so that there is no need to fetch the customer frequently. What is included in context:

1. User is Logged in

Context consists of three main parts :

- Customer Details (Columns: age, gender, size, cust_id). The system fetches data automatically from CSVs using pandas.
- Transaction History (Columns: t_dat, SKU_No, payment_method, prod_return_type / return_Type, return_Status, delivered)

- Product Details (Columns: SKU_No, prod_name, product_type_name, product_group_name,
  colour_group_name, department_name, Price_INR, Discount_Percentage, Return_Type, Eco_Score)
- Valid customer IDs – 1,3,5.

2. User is logged out

- The LLM must explicitly ask for age and gender before proceeding.
  Once that information is provided, a minimal context is created using those values.

AI Stylist:

The **AI Stylist** is responsible for understanding the user's fashion intent and recommending the most relevant clothing products. It personalizes results using the **customer's context** (age, gender, purchase history, etc.) and **semantic understanding** of the user query. Its done in the following 3 3 steps:

1. Query Refinement (refine_prompt)

The user query and customer context are passed to a dedicated LLM prompt called **refine_prompt**. Its purpose is to **rewrite the user query** into a refined, detailed version that takes into account user-specific attributes.

Example:

```
Gender: Female
Age: 25-30
Postal Code: Mumbai
--- PURCHASE TRANSACTIONS ---
Transaction 0: Purchased formal shirts and trousers
--- PRODUCTS PURCHASED ---
Product 0: Cotton Slim Shirt, Color: Blue
Product 1: Linen Regular Fit Pant, Color: Beige

User Query:
show me something to wear to a party

Refined Query:
show me trendy but elegant party outfits for a 25-year-old female from Mumbai,
preferably in styles similar to her past purchases like shirts and pants, in blue or beige tones.
```

2. Searching Product Catalog (search_products)

The **refined query** and all product descriptions are converted into **numeric embeddings** using the **BGE model**.

Then the system:

1. Computes **cosine similarity** between the query embedding and every product embedding.
2. Measures how semantically close each product is to the query intent.
3. Sorts all products by similarity score.
4. Returns the **top 20 most relevant products**.

This ensures that results are driven by **semantic meaning**, not just keywords.

3. Ranking the Top 3 (recommend_top3_structured)

The top 20 products and the refined query are passed to another LLM called recommend_top3_structured, whose goal is to identify the top 3 most suitable products for the user.

Weightage Logic:

- 80% → Current user query meaning (highest priority).
- 15% → Customer context (e.g., gender, location, preferences).
- 5% → Purchase history (style consistency only).

This prevents bias from past purchases — e.g., a user who bought party wear earlier but now asks for office wear will get relevant new options.

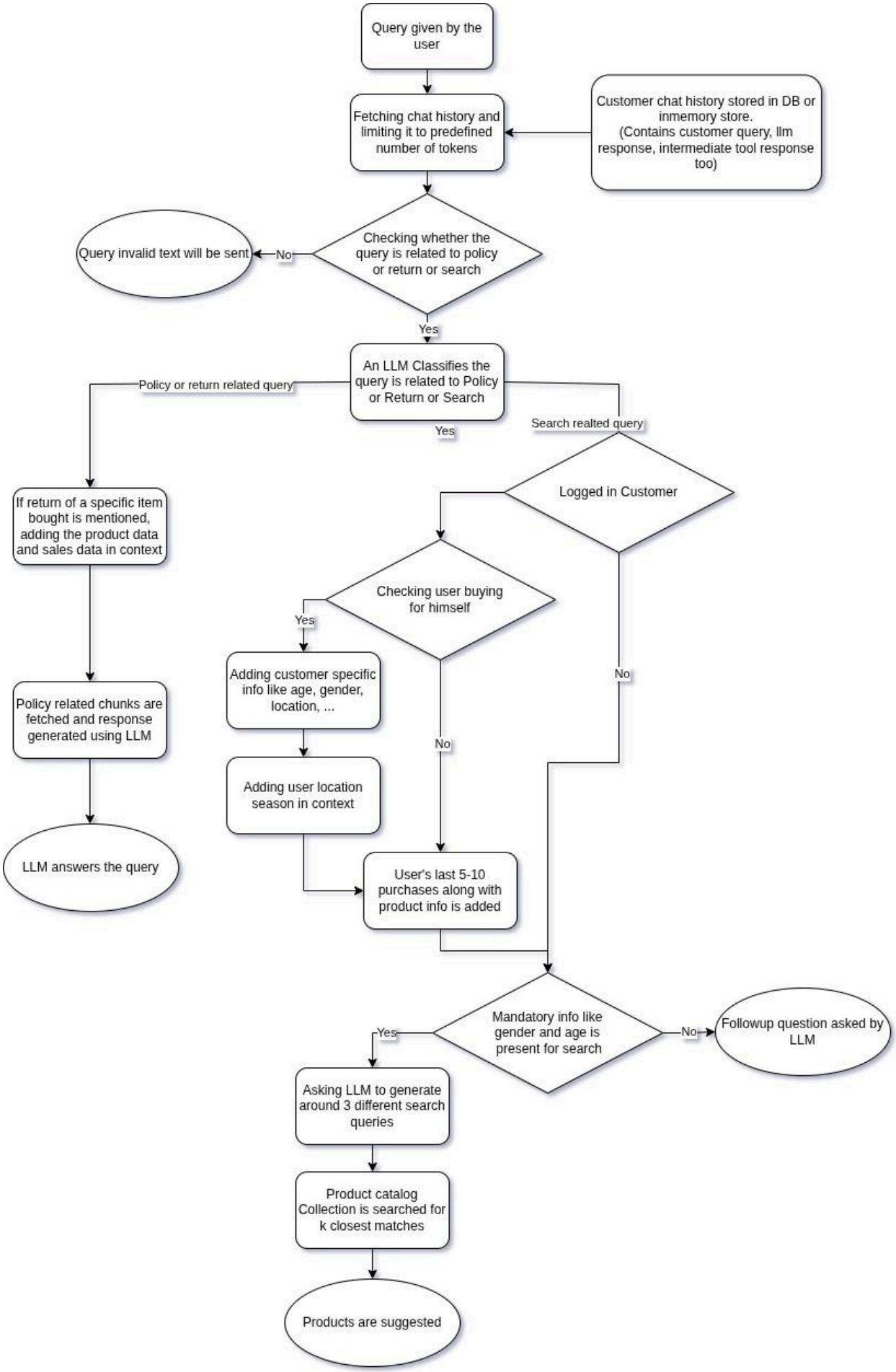
The LLM outputs the final top 3 products in a structured format.

Policy LLM:

The **Policy LLM** handles all customer queries related to **store policies, returns, exchanges, and refunds**. It ensures accurate, personalized answers by combining stored policy data, and conversation history. Its does the following things:

- The model identifies the **policy theme** of the user query (e.g., “return window,” “exchange,” “refund timeline”).
- It retrieves the **relevant policy section** from stored data or rules.
- The response is then generated in **natural, conversational form**, maintaining accuracy and empathy.

Flow chart: (accommodated in the next page)


Query given by the user
Fetching chat history and limiting it to predefined number of tokens
Customer chat history stored in DB or inmemory store. (Contains customer query, llm response, intermediate tool response too)
Checking whether the query is related to policy or return or search
No
Query invalid text will be sent
Yes
An LLM Classifies the query is related to Policy or Return or Search
Policy or return related query
Yes
Search realted query
If return of a specific item bought is mentioned, adding the product data and sales data in context
Logged in Customer
Checking user buying for himself
Yes
No
Adding customer specific info like age, gender, location, ...
Policy related chunks are fetched and response generated using LLM
Adding user location season in context
No
LLM answers the query
User's last 5-10 purchases along with product info is added
Mandatory info like gender and age is present for search
Yes
No
Followup question asked by LLM
Asking LLM to generate around 3 different search queries
Product catalog Collection is searched for k closest matches
Products are suggested

Hyperparameter Tuning for Policy RAG

Parameters chosen:

1. Model: Embedder which is used to embed the doc
2. Chunk size: The max word length of chunks that will be embedded
3. Overlap: The number of overlapping words between consecutive chunks
4. Retrieval strategy: One collection will be used / two

Combination 1:

```
Model: BAAI/bge-base-en-v1.5
Chunk size: 700 (collection 1), 300 (collection 2)
Overlap: 70 (collection 1), 30 (collection 2)
Retrieval: Both collections are queried and chunks with high cosine similarity is chosen.
Comparison done on results from both collection
```

Combination 2:

```
Model: all-MiniLM-L6-v2
Chunk size: 700
Overlap: 70
Retrieval: normal one collection setup
```

Scoring:

We are doing manual scoring at this stage. We have the information about the availability of context for the test queries and check if context is fetched properly. Also we finally check whether the LLM answers properly using the provided context.

Observations:

Scoring is out of 5

Query	combination 1	combination 2
1	5	3
2	4.5	4.5
3	5	4
4	5	3
5	4.5	0
Average	4.8	2.9

- BAAI/bge-base-en-v1.5 performed well.
- Regarding finding out invalid queries, our RAG pipeline performed well without any need for hyperparameter tuning. Also, 100% of the time it found out when age or gender were missing.
- Please refer to this sheet for more information.

Sample test result:

--- Query 2 ---

Query: can i get discount voucher if i return old clothes

Context: ng way to recycle their used garments. By returning used clothes to our stores for responsible recycling or repurposing, customers contribute to lowering carbon emissions, conserving natural resources, and supporting a circular fashion economy. 2. Policy Overview Customers are invited to bring any brand of used or unwanted clothing, shoes, or textiles (in any condition) to participating stores. In return, they will receive a 15% discount voucher applicable toward the purchase of new clothing on their next visit. 3. How It Works 1. DropOff: Customers may drop their used textiles at designated Recycling Collection Boxes located near store entrances or customer service desks. Processed for the product amount only. The following are nonrefundable: ● Shipping charges ● Platform convenience fees ● Giftwrapping or special

service charges However, if a product is damaged or defective due to Clothing Retail's fault, shipping or convenience fees may be refunded upon request within 7 days of refund credit. 5. Refund Conditions Refunds will be approved only when: ● The returned product is in its original condition, with tags, packaging, and invoice intact. ● The item passed a quality inspection at our warehouse. ● No damage, alteration, or signs of usage are found postdelivery. ● Accessories or bundled items (if applicable) are returned together. Any returns will not be accepted. ● For electronics or watches, refunds are processed after the item passes quality inspection at the brand warehouse. Failure to comply with the above may result in rejection of the return request. 5. Return Process 1. Log in to your Clothing Retail account. 2. Go to "My Orders" and click on Return/Exchange for the respective item. 3. Choose Return or Exchange, specify the reason, and upload supporting images (if damaged). 4. Select your preferred refund mode (original payment method or Clothing Retail Wallet). 5. A pickup will be scheduled within 4–7 days or instructions for selfshipping will be provided. 6. Once the product passes the qual accessories or bundled items (if applicable) are returned together. Any product found in violation of these conditions will be deemed ineligible for refund. 6. Refund for NonReturnable Products Certain categories are nonreturnable (e.g., innerwear, accessories, jewelry, or electronics marked "NonReturnable"). Refunds for these will only be approved if the product was: ● Defective, ● Damaged in transit, or ● Incorrectly shipped. Such refunds are processed after internal verification and approval by our quality team. 7. Refund Notifications You will be kept informed through: ● Email notifications ● SMS updates ● Order status in your Clothing Retail Account If there's case pickup is unavailable at your location, you may selfship the product to our warehouse; the shipping cost will be reimbursed as Clothing Retail credits, subject to policy compliance. ● Returned products must be unused, unwashed, unaltered, and in their original condition with all tags,...

Response: Yes, if you return used or unwanted clothing, shoes, or textiles to participating stores, you will receive a 15% discount voucher applicable toward the purchase of new clothing on your next visit.

Please refer to this file for policy rag for full logs

Hyperparameter Tuning for Product RAG

Parameters chosen:

1. Temperature: LLM's creativity
2. Top-K: Before reranking and choosing best 3 how many products should be fetched from product catalog
3. Search Query Refiner prompt: Search query given by the user may not be complete. With the context available LLM will refine the search query
4. Reranker Prompt: The results we get from product RAG follows cosine similarity and it need not be correct. So, an LLM performs reranking on a large set and suggest a small set of well curated products

Combination 1:

```
Temp: 0.2
Top-K: 5
Refine Prompt: refine_prompt_contextual
Rerank Prompt: rerank_prompt_contextual
```

Combination 2:

```
Temp: 0
Top-K: 10
Refine Prompt: refine_prompt_precise
Rerank Prompt: rerank_prompt_precise
```

Combination 3:

```
Temp: 0.5
Top-K: 12
Refine Prompt: refine_prompt_balanced
Rerank Prompt: rerank_prompt_balanced
```

Combination 4:

```
Temp: 0.8
Top-K: 20
Refine Prompt: refine_prompt_creative
Rerank Prompt: rerank_prompt_creative
```

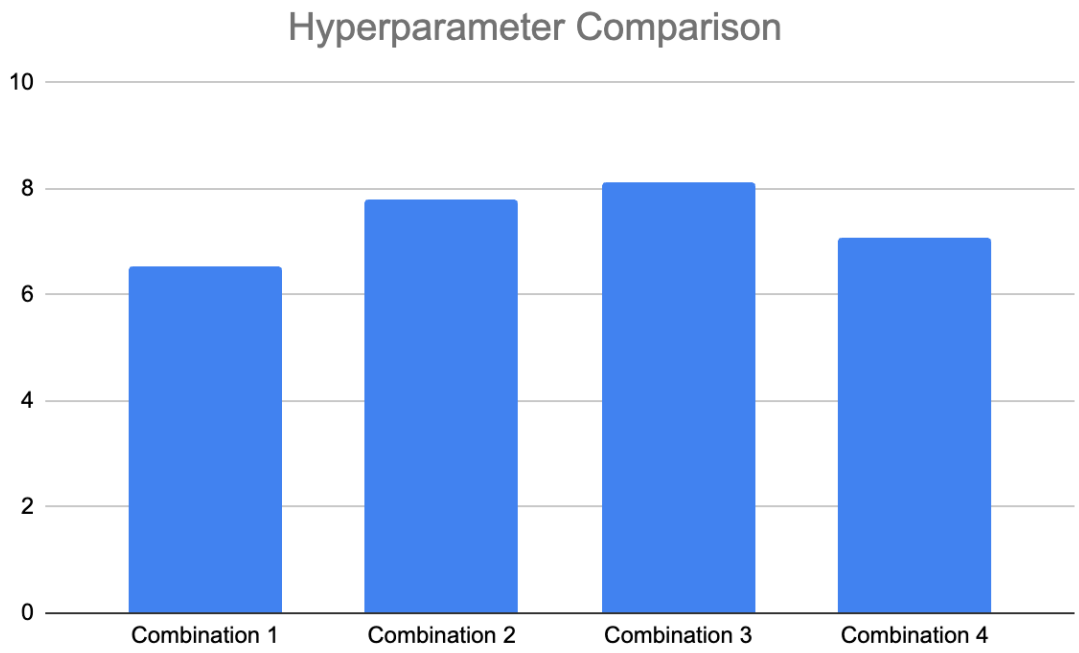
Scoring:

The chat bot results in three product suggestions. So, the setup is we rated the three product suggestions and the average of the three suggestions we will use as query score and use the same to compare with other hyper parameter combinations

We performed scoring manually on the results produced with different sets of hyperparameter configurations. Also we tried generating scores using LLM to get a better picture. But for final comparisons we will use manual scores as it is more reliable.

Observations:

Scoring is out of 10



From the graph we can see that combination 3 is performing better. So, when both creativity and precision are given importance, the RAG pipeline performed well.

```
Temp: 0.5
Top-K: 12
Refine Prompt: refine_prompt_balanced
Rerank Prompt: rerank_prompt_balanced
```

Please refer this sheet for more information

Sample test result:

```
Temp: 0.2
Top-K: 5
Refine Prompt: refine_prompt_contextual
Rerank Prompt: rerank_prompt_contextual
Query1:

Product 1:
• Name: SAILOR FRONT PRINT
• Type: Sweater
• Color: Red
• Price: 3821 INR
• Discount: 28%
• Return Type: 7 days return
• SKU: N/A

Product 2:
• Name: CORN CLASSIC TEE
• Type: T-shirt
• Color: Dark Grey
• Price: 3157 INR
```

```
• Discount: 24%
• Return Type: No return
• SKU: N/A

Product 3:
• Name: SPEED Paprika
• Type: Sweater
• Color: Grey
• Price: 2533 INR
• Discount: 53%
• Return Type: 15 days return
• SKU: N/A
```

The same way we run and document the results. Please refer to this document for full runtime logs. To see various prompts used please to this package.

Summary of previous Milestones

Milestone 1:

- This milestone consists of the problem definition and literature review. The project objective was divided into two parts, A and B. The first one being “Store policy + Customer help desk” and the other part is an “AI Stylist”.
- It then talks about the existing solutions, baselines and benchmarks. Taking a deep dive into them.
- Concluding that our suggestion system is going to use text features alone and will suggest clothes based on user inputs.
- The folder structure in as given below:
    - /group-5-DS-AI-Lab-Project
        - /milestones
            - /milestone-1
                - Milestone - 1.pdf

Milestone 2:

- This milestone had details about us choosing a dataset, and performing analysis on it.
- It had some sample images from the image dataset.
- The code for the EDA performed is also provided in a jupyter notebook.
- The FAQ and Policy details are provided in a pdf file with the same name.
- And the dataset for articles, customers detail and customers' purchase histories have been shared.
- The folder structure is as given below:
    - /group-5-DS-AI-Lab-Project
        - /milestones
            - /milestone-2
                - /sample_images
                    - (a few images are here)
                - EDA_on_catalog.ipynb
                - Milestone - 2.pdf
                - articles.csv
                - customers.csv
                - faq_and_policy.pdf
                - purchase_history.xlsx

Milestone 3:

- The third milestone had intricate details about the RAG (Retrieval Augmented Generation) method and a handful of embedding models to compare their strengths and weaknesses.
- A jupyter notebook showing the hands-on code for the whole embedding pipeline.
- It starts the work with "filtered_data_with_images.csv" containing the textual details about the images, and embeds it using the "all-MiniLM-L6-v2" model.
- These vectors and the meta data are then saved for later use.
- And, then using the FAISS (Facebook AI Similarity Search) to deal with the search queries.
- Finally concluding with some sample queries and their results.
- The folder structure is as given below:
    - /group-5-DS-AI-Lab-Project
        - /milestones
            - /milestone-3
                - EmbeddingTest.ipynb
                - Milestone - 3.pdf